

Fix submdspan for C++26

Document #: P3355R1
Date: 2024-10-15
Project: Programming Language C++
Library Evolution
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>

Contents

- [1 Revision History](#)
- [2 Abstract](#)
- [3 Motivation and design discussion](#)
 - [3.1 Vectorization example](#)
 - [3.2 Issues with C++ Working Draft](#)
 - [3.3 What this proposal does and doesn't change](#)
 - [3.4 Why we don't try to fix `aligned\_accessor::offset`](#)
- [4 Implementation experience](#)
- [5 Desired ship vehicle](#)
- [6 Wording](#)
 - [6.1 Increment `\_\_cpp\_lib\_submdspan` feature test macro](#)
 - [6.2 Change existing layout mappings' `submdspan\_mapping` results](#)

§ 1 Revision History

- Revision 0 submitted 2024-07-14
 - LEWG reviewed this version 2024-10-15, and voted to forward the [mdspan.sub] changes (with the [mdspan.syn], i.e., the “accept user-defined pair types” part, removed) to LWG. LEWG also asked that we increment the submdspan feature test macro `__cpp_lib_submdspan` from its current value (202403L, set by adoption of P2642R6 (“Padded mdspan layouts”) into the Working Draft for C++26).
- Revision 1 to be submitted for the 2024-10-16 mailing

- Remove “user-defined pair types as slices” feature. We will make this a separate paper.
- Increment `__cpp_lib_submdspan` from its current value (202403L).
- Implement suggestion during LEWG wording review to define a “wording macro” for all the slice specifier types that have unit stride. We call it *unit-stride slice for M*, where M is a layout mapping type. (The definition only depends on an `index_type`, but the wording is most natural when it depends on the layout mapping type for which *submdspan-mapping-impl* is a private member function.)

§ 2 Abstract

We propose to change `submdspan_mapping` for the following layouts’ layout mappings:

- `layout_left`,
- `layout_right`,
- `layout_left_padded`, and
- `layout_right_padded`,

so that a `strided_slice` slice with compile-time unit stride results in the returned mapping having the same layout as if the slice were a pair of integers. This preserves compile-time optimization information for common layouts.

This change needs to be merged into the Working Draft before C++26. Otherwise, it would be a breaking change.

§ 3 Motivation and design discussion

§ 3.1 Vectorization example

Suppose that one wants to vectorize a 1-D array copy operation using `mdspan` and `aligned_accessor` (P2897). One has a `copy_8_floats` function that optimizes the special case of copying a contiguous array of 8 floats, where the start of the array is aligned to `8 * sizeof(float)` (32) bytes. In practice, plenty of libraries exist to optimize 1-D array copy. This is just an example that simplifies the use cases for explicit 8-wide SIMD enough to show in a brief proposal.

```
template<class ElementType, size_t ext, size_t byte_alignment>
using aligned_array_view = mdspan<ElementType,
    extents<int, ext>, layout_right,
    aligned_accessor<ElementType, byte_alignment>>;

void
copy_8_floats(aligned_array_view<const float, 8, 32> src,
```

```

        aligned_array_view<float, 8, 32> dst)
    {
        // One would instead use a hardware instruction for aligned
copy,
        // or a "simd" or "unroll" pragma.
        for (int k = 0; k < 8; ++k) {
            dst[k] = src[k];
        }
    }
}

```

The natural next step would be to use `copy_8_floats` to implement copying 1-D float arrays by the usual “strip-mining” approach.

```

template<class ElementType>
using array_view = mdspan<ElementType, dims<1, int>>;

void slow_copy(array_view<const float> src, array_view<float> dst)
{
    assert(src.extent(0) == dst.extent(0));
    for (int k = 0; k < src.extent(0); ++k) {
        dst[k] = src[k];
    }
}

template<size_t vector_length>
void strip_mined_copy(
    aligned_array_view<const float, dynamic_extent,
        vector_length * sizeof(float)> src,
    aligned_array_view<float, dynamic_extent,
        vector_length * sizeof(float)> dst)
{
    assert(src.extent(0) == dst.extent(0));
    assert(src.extent(0) % vector_length == 0);

    for (int beg = 0; beg < src.extent(0); beg += vector_length) {
        constexpr auto one = std::integral_constant<int, 1>{};
        constexpr auto vec_len = std::integral_constant<int,
vector_length>{};

        // Using strided_slice lets the extent be a compile-time
constant.
        // tuple{beg, beg + vector_length} would result in
dynamic_extent.
        constexpr auto vector_slice =
            strided_slice{.offset=dst_lower, .extent=vector_length,
        .stride=one};

        // PROBLEM: Current wording makes this always layout_stride,
// but we know that it could be layout_right.
        auto src_slice = submdspan(src, vector_slice);
        auto dst_slice = submdspan(dst, vector_slice);

        copy_8_floats(src_slice, dst_slice);
    }
}

```

```

void copy(array_view<const float> src, array_view<float> dst)
{
    assert(src.extent(0) == dst.extent(0));
    constexpr int vector_length = 8;

    // Handle possibly unaligned prefix of less than vector_length
    elements.
    auto aligned_starting_index = [](auto* ptr) {
        constexpr auto v = static_cast<unsigned>(vector_length);
        auto ptr_value = reinterpret_cast<uintptr_t>(ptr_value);
        auto remainder = ptr_value % v;
        return static_cast<int>(ptr_value + (v - remainder) % v);
    };
    int src_beg = aligned_starting_index(src.data());
    int dst_beg = aligned_starting_index(dst.data());
    if (src_beg != dst_beg) {
        slow_copy(src, dst);
        return;
    }
    slow_copy(submdspan(src, tuple{0, src_beg}),
              submdspan(dst, tuple{0, dst_beg}));

    // Strip-mine the aligned vector_length segments of the array.
    int src_end = (src.size() / vector_length) * vector_length;
    int dst_end = (dst.size() / vector_length) * vector_length;
    strip_mined_copy<8>(submdspan(src, tuple{src_beg, src_end}),
                       submdspan(dst, tuple{dst_beg, dst_end}));

    // Handle suffix of less than vector_length elements.
    slow_copy(submdspan(src, tuple{src_end, src.extent(0)}),
              submdspan(dst, tuple{dst_end, dst.extent(0)}));
}

```

The `strip_mined_copy` function must use `strided_slice` to get slices of 8 elements at a time, rather than `tuple`. This ensures that the resulting extent is a compile-time constant 8, even though the slice starts at a run-time index `beg`.

§ 3.2 Issues with C++ Working Draft

The current C++ Working Draft has two issues that hinder optimization of the above code.

1. The above `submdspan` results always have `layout_stride`, even though we know that they are contiguous and thus should have `layout_right`.
2. The `submdspan` operations in `strip_mined_copy` should result in `aligned_accessor` with 32-byte alignment, but instead give `default_accessor`. This is because `aligned_accessor`'s `offset` member function takes the offset as a `size_t`. This discards compile-time information, namely that the offset can be expressed as the product of some integer and the overalignment factor, where the overalignment factor is known at compile time.

§ 3.3 What this proposal does and doesn't change

This proposal fixes (1) for all layouts currently in the Working Draft that have a `submdspan_mapping` customization: `layout_left`, `layout_right`, `layout_left_padded`, and `layout_right_padded`. We can do that without breaking changes, as long as this proposal is merged before C++26 is finalized. After that, merging the proposal would be a breaking change.

This proposal does *not* fix (2), because that would require a breaking change to both the layout mapping requirements and the accessor requirements, and because it would complicate both of them quite a bit.

§ 3.4 Why we don't try to fix `aligned_accessor::offset`

Regarding (2), [\[mdspan.submdspan.submdspan\] 6](#) says that `submdspan(src, slices...)` has effects equivalent to the following.

```
auto sub_map_offset = submdspan_mapping(src.mapping(), slices...);
return mdspan(src.accessor().offset(src.data(),
sub_map_offset.offset),
            sub_map_offset.mapping,
            AccessorPolicy::offset_policy(src.accessor()));
```

The problem is `AccessorPolicy::offset_policy(src.accessor())`. The type `offset_policy` is the wrong type in this case, `default_accessor<const float>` instead of `aligned_accessor<const float, 32>`. If we want an offset with suitable compile-time alignment to have a different accessor type, then we would need at least the following changes.

1. The Standard Library would need a new type that represents the product of a compile-time integer (that is, an *integral-constant-like* type) and a “run-time” integer (an *integral-not-bool* type). It would need overloaded arithmetic operators that preserve this product form as much as possible. For example, $8x + 4$ for a run-time integer x should result in $4y$ where $y = 2x + 1$ is a run-time integer.
2. At least the Standard layout mappings' `operator()` would need to compute with these types and return them if possible. The layout mapping requirements would thus need to change, as currently `operator()` must return `index_type` (see [\[\[mdspan.layout.reqmts\]\] 7](#)).
3. `aligned_accessor::offset` would need an overload taking a type that expresses the product of a compile-time integer (of suitable alignment) and a run-time integer. The accessor requirements [\[\[mdspan.accessor.reqmts\]\]](#) may also need to change to permit this.
4. The definition of `submdspan` would need some way to get the accessor type corresponding to the new offset overload, instead of `aligned_accessor::offset_policy` (which in this case is `default_accessor`).

The work-around is to convert the result of `submdspan` by hand to use the desired accessor. In the above copy example, one would replace the line

```
copy_8_floats(src_slice, dst_slice);
```

with the following, that depends on `aligned_accessor`'s explicit constructor from `default_accessor`.

```
copy_8_floats(aligned_array_view<const float, 8, 32>{src},
              aligned_array_view<float, 8, 32>{dst});
```

Given that this work-around is easy to do, should only be needed for a few special cases, and avoids a big design change to the accessor policy requirements, we don't propose trying to fix this issue in the C++ Working Draft.

§ 4 Implementation experience

Daisy Hollman's original implementation of `submdspan` implemented strided slices in this way.

§ 5 Desired ship vehicle

C++26 / IS.

§ 6 Wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

§ 6.1 Increment `__cpp_lib_submdspan` feature test macro

In `[version.syn]`, increase the value of the `__cpp_lib_submdspan` macro by replacing `YYMMML` below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_submdspan YYMMML // also in <mdspan>
```

§ 6.2 Change existing layout mappings' `submdspan_mapping` results

Append the following to the end of `[mdspan.sub.map.common]`.

⁹ Given a layout mapping type `M`, a type `S` is a *unit-stride slice for M* if

(9.1) `S` is a specialization of `strided_slice` where `S::stride_type` models *integral-constant-like* and `S::stride_type::value` equals 1,

(9.2) S models *index-pair-like* $\langle M::\text{index_type} \rangle$, or

(9.3) *is_convertible_v* $\langle S, \text{full_extent_t} \rangle$ is true.

Throughout **[mdspan.sub]**, wherever the text says

“ S_k models *index-pair-like* $\langle \text{index_type} \rangle$ or *is_convertible_v* $\langle S_k, \text{full_extent_t} \rangle$ is true,”

replace it with

“ S_k is a unit-stride slice for `decltype(*this)`.”

Apply the analogous transformation if the text says S_p or S_0 , but is otherwise the same. Make this set of changes in the following places.

- **[mdspan.sub.map.left]** (1.3.2), (1.4), (1.4.1), and (1.4.3);
- **[mdspan.sub.map.right]** (1.3.2), (1.4), (1.4.1), and (1.4.3);
- **[mdspan.sub.map.leftpad]** (1.3.2), (1.4), (1.4.1), and (1.4.3); and
- **[mdspan.sub.map.rightpad]** (1.3.2), (1.4), (1.4.1), and (1.4.3).